Übung 7: Graphen I und II

(Dauer: 90 Minuten - Gruppenarbeit empfohlen)

Ziel der Übung: Sie lernen, Problemstellungen als Graphen zu modellieren und Graphen mit Breitensuche (BFS), Tiefensuche (DFS) und Dijkstra systematisch zu traversieren bzw. kürzeste Pfade und Erreichbarkeit zu bestimmen. Sie wenden diese Algorithmen auf verschiedene praktische Problemstellungen an und analysieren ihre Eigenschaften und Laufzeiten.

1. Betrachten Sie den folgenden ungerichteten, ungewichteten Graphen:

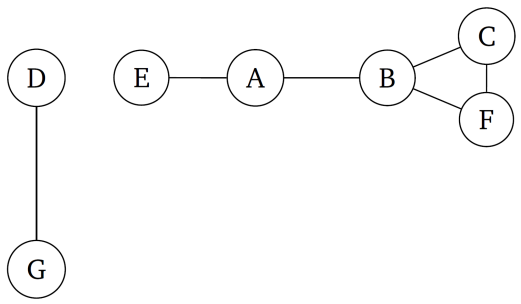


Abbildung 1: ungerichteter, ungewichteter Graph

   (a) Geben Sie die Knotenmenge V und die Kantenmenge E des Graphen an. Bestimmen Sie außerdem die Anzahl der Knoten $|V|$ und die Anzahl der Kanten $|E|$.

   (b) Wie groß ist der maximale Knotengrad des Graphen?

   (c) Enthält der Graph einen Zyklus? Falls ja, geben Sie einen Zyklus an.

   (d) Geben Sie einen einfachen Pfad maximaler Länge an. Hinweis: Die Länge eines Pfades bezeichnet die Anzahl seiner Kanten.

   (e) Welche Kante könnten Sie dem Graphen hinzufügen, sodass der längste einfache Pfad des entstehenden Graphen die Länge 6 hat?

   (f) Bestimmen Sie die Zusammenhangskomponenten des Graphen.

2. Betrachten Sie den folgenden ungerichteten, gewichteten Graphen:

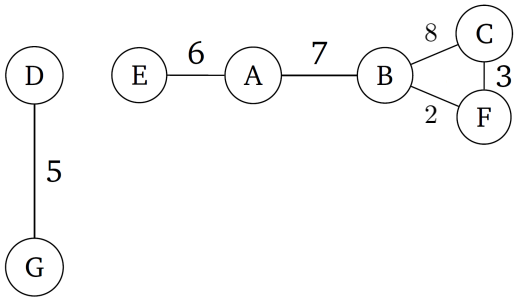


Abbildung 2: ungerichteter, gewichteter Graph

   (a) Welcher Pfad von E nach C enthält die geringste Anzahl an Knoten? Geben Sie auch die Anzahl der Knoten dieses Pfades an.

   (b) Welcher Pfad von E nach C hat die geringsten Kosten? Geben Sie die Kosten dieses Pfades an.

   (c) Welcher Pfad von E nach C hat die geringste Länge? Geben Sie die Länge dieses Pfades an.

3. Betrachten Sie den folgenden ungerichteten, ungewichteten Graphen. Wir beginnen die Traversierung beim Knoten A. Es sollen zwei mögliche Besuchsreihenfolgen für eine Breitensuche (BFS) und eine Tiefensuche (DFS), die beim Knoten A beginnen, angegeben werden.

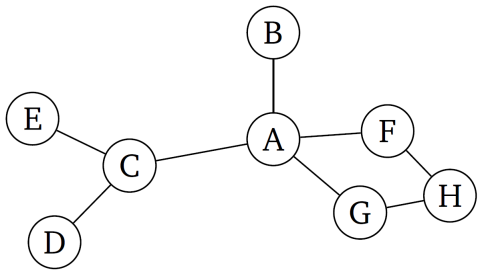


Abbildung 3: ungerichteter, ungewichteter Graph

   (a) Bestimmen Sie für jeweils eine der BFS- und DFS-Traversierungen die Besuchsreihenfolge, indem Sie den jeweiligen Algorithmus Schritt für Schritt ausführen. Dokumentieren Sie dazu in jeder Iteration der äußeren `while`-Schleife den Inhalt der verwendeten Warteschlange (bei BFS) bzw. des verwendeten Stapels (bei DFS).

   (b) Bestimmen Sie jeweils eine weitere mögliche Besuchsreihenfolge, ohne den Algorithmus Schritt für Schritt auszuführen.

4. Betrachten Sie den folgenden gerichteten, ungewichteten Graphen:

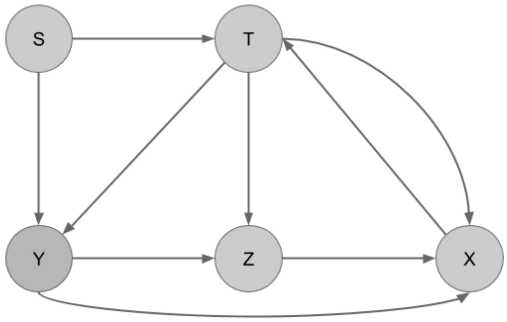


Abbildung 4: gerichteter, ungewichteter Graph

   Führen Sie eine Breitensuche (BFS) ausgehend vom Knoten S durch, um die kürzesten Pfade von S zu allen erreichbaren Knoten zu bestimmen.

   (a) Tragen Sie für jeden Knoten seinen Vorgängerknoten in die folgende Tabelle ein. Kennzeichnen Sie einen Knoten, sobald dieser in `visited` gespeichert ist.

   (b) Zeichnen Sie nach Abschluss der Breitensuche den resultierenden Baum der kürzesten Pfade.

   Hinweis: Wenn mehrere Knoten zur Auswahl stehen, betrachten Sie die Knoten in alphabetischer Reihenfolge.

Knoten	Vorgängerknoten	In visited gespeichert?
S		
T		
X		
Y		
Z		

5. Betrachten Sie den folgenden gerichteten, gewichteten Graphen:

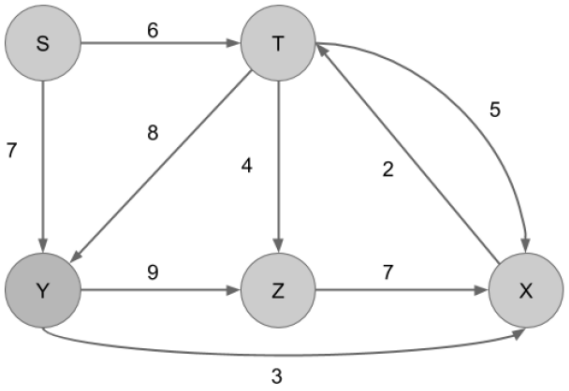


Abbildung 5: gerichteter, gewichteter Graph

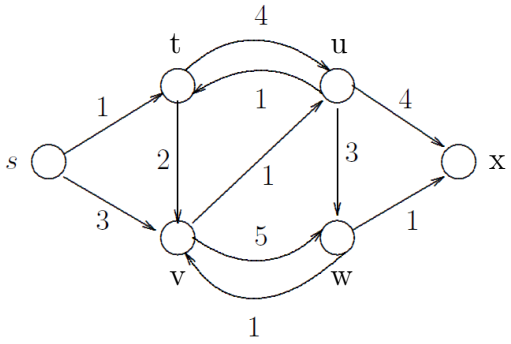
Führen Sie den Dijkstra-Algorithmus ausgehend vom Startknoten S aus. Verwenden Sie die folgende Tabelle, um die einzelnen Schritte des Algorithmus zu dokumentieren.

Tragen Sie für jeden Knoten die aktuell bekannte Distanz zum Startknoten, seinen Vorgängerknoten und den Zeitpunkt ein, zu dem der Knoten verarbeitet wird.

Zeichnen Sie anschließend den resultierenden Baum kürzester Pfade.

Knoten	Distanz zu Startknoten	Vorgängerknoten	Verarbeitet?
S			
T			
X			
Y			
Z			

6. Kürzeste Pfade in einem gewichteten Graphen.



(a) Bestimmen Sie die kürzesten Distanzen von s zu allen anderen Knoten. Tragen Sie die endgültigen Distanzen und die jeweiligen Vorgängerknoten in die Tabelle ein.

Knoten	Distanz	Vorgänger	Verarbeitet?
s			
t			
u			
v			
w			
x			

(b) Zeichnen Sie den Baum kürzester Pfade mit Startknoten s.

7. Sie entwerfen ein 2D-Top-Down-Videospiel, dessen Welt als Gitter dargestellt wird. Jedes Feld ist entweder eine unpassierbare Wand, ein begehbares Feld oder ein Wurmloch. In einem Zug kann sich der Spieler auf ein orthogonal benachbartes begehbares Feld bewegen. Befindet sich der Spieler auf einem Wurmloch, kann er sich alternativ in einem Zug zum anderen Ende des Wurmlochs teleportieren.

   Auf dem Spielfeld befinden sich mehrere Münzen. Gesucht ist ein Pfad vom Startfeld des Spielers zu einer Münze, der möglichst wenige Züge benötigt.

   (a) Modellieren Sie das Szenario als Graphen. Beschreiben Sie, was die Knoten und Kanten darstellen und ob der Graph gerichtet bzw. ungerichtet sowie gewichtet bzw. ungewichtet ist.

   (b) Welchen Algorithmus würden Sie verwenden, um einen Pfad mit minimaler Anzahl an Zügen zu einer Münze zu finden? Begründen Sie Ihre Wahl.

8. Sie haben gerade Ihren ersten großen DJ-Auftritt als "DJ Kistra" ergattert. Während Ihrer Show spielen Sie "Shake It Off" und möchten anschließend zu "Wildest Dreams" übergehen.

   Das Publikum soll nicht enttäuscht werden, wenn zwei aufeinanderfolgende Songs einen BPM-Unterschied von mehr als 10 haben. Außerdem muss jeder Song vollständig abgespielt werden.

   Sie möchten daher eine Folge von Songs finden, die mit "Shake It Off" beginnt und mit "Wildest Dreams" endet. Für zwei aufeinanderfolgende Songs darf der BPM-Unterschied höchstens 10 betragen, und die Songs sollen schrittweise langsamer werden.

   Gesucht ist eine solche Songfolge mit minimaler Gesamtdauer. Für jeden Song sind seine Geschwindigkeit in Beats per Minute (BPM) und seine Dauer in Sekunden gegeben.

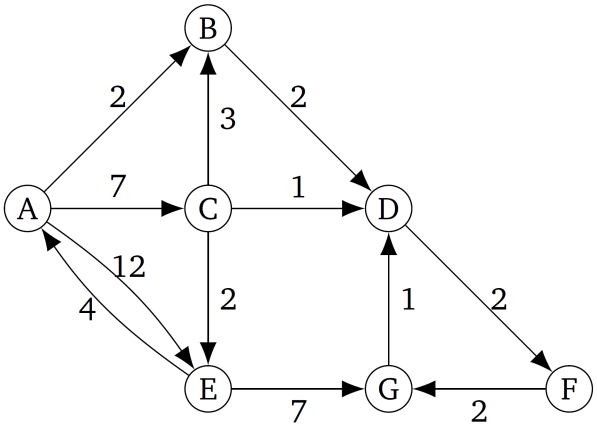
   (a) Beschreiben Sie einen gerichteten, gewichteten Graphen, mit dem Sie dieses Problem modellieren können. Geben Sie insbesondere an, was die Knoten und Kanten darstellen und wie die Kantengewichte definiert werden.

   (b) Beschreiben Sie einen Algorithmus, mit dem Sie diesen Graphen konstruieren können. Sie können davon ausgehen, dass die Songs in einer geeigneten Datenstruktur gespeichert sind. Verwenden Sie die Methode `makeEdge(v1,v2,w)`, um eine gerichtete Kante von v_1 nach v_2 mit Gewicht w zu erzeugen.

   (c) Welchen Algorithmus können Sie auf dem konstruierten Graphen anwenden, um eine zulässige Songfolge mit minimaler Gesamtdauer zu finden?

   (d) Bestimmen Sie die Laufzeit Ihres Verfahrens. Geben Sie sowohl die Laufzeit für die Konstruktion des Graphen als auch für die Suche nach der optimalen Songfolge an.

9. Kürzeste Pfade in einem gewichteten Graphen.



(a) Bestimmen Sie den kürzesten Pfad vom Startknoten A aus zu allen anderen Knoten des obigen Graphen. Sie können dazu folgende Tabelle nutzen. Die kürzesten Abstände vom Knoten A aus sollen am Ende in der Spalte Distanz stehen.

Knoten	Distanz	Vorgänger	Verarbeitet?
A			
B			
C			
D			
E			
F			
G			

(b) Zeichnen Sie den Baum kürzester Pfade von Startknoten A aus.

10. Ein soziales Netzwerk möchte einem Nutzer neue Freundschaften vorschlagen. Dazu sollen insbesondere solche Nutzer vorgeschlagen werden, die mit möglichst vielen Freunden des ursprünglichen Nutzers befreundet sind.

    Entwerfen Sie einen Algorithmus, der für einen gegebenen Nutzer die Anzahl der gemeinsamen Freunde für mögliche neue Freunde bestimmt und die möglichen neuen Freunde nach dieser Anzahl sortiert.

    (a) Beschreiben Sie Ihren Algorithmus in Pseudocode oder in Worten.

    (b) Welche Datenstruktur verwenden Sie, um für jeden möglichen neuen Freund die Anzahl der gemeinsamen Freunde zu speichern? Begründen Sie Ihre Wahl kurz.

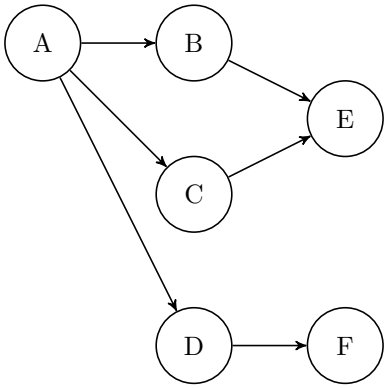


Abbildung 6: Knoten A ist der Nutzer, dem neue Freunde vorgeschlagen werden sollen. B, C und D sind bereits mit A befreundet. E ist mit zwei Freunden von A befreundet und erhält daher eine höhere Empfehlung als F, das nur mit einem Freund von A befreundet ist.

11. Erweitern Sie den folgenden iterativen DFS-Algorithmus so, dass er das Erreichbarkeitsproblem löst.

    Gegeben seien ein Startknoten s und ein Zielknoten t. Der Algorithmus soll `True` zurückgeben, wenn ein Pfad von s nach t existiert, und `False`, wenn kein solcher Pfad existiert.

Algorithm 1 Iterative Tiefensuche (DFS)

```
1:  function dfs(Graph graph, Vertex start) {
2:    Stack<Vertex> perimeter = new Stack<>();
3:    Set<Vertex> visited = new Set<>();
4:
5:    perimeter.add(start);
6:
7:    while (!perimeter.isEmpty()) {
8:      Vertex from = perimeter.remove();
9:      if (!visited.contains(from)) {
10:       for (Edge edge : graph.edgesFrom(from)) {
11:         Vertex to = edge.to();
12:         if (!visited.contains(to))
13:           perimeter.add(to);
14:       }
15:       visited.add(from);
16:     }
17:   }
18: }
```

Aufgabe: Geben Sie an, an welchen Stellen des gegebenen Algorithmus Änderungen vorgenommen werden müssen. Geben Sie den zusätzlich benötigten Code als Pseudocode an. Eine vollständige Implementierung des Algorithmus ist nicht erforderlich.

Begründen Sie kurz, warum Ihre Änderungen das Erreichbarkeitsproblem lösen.

12. Modellieren Sie ein Rezept zum Kuchen backen als Graph und geben Sie die Mindestdauer für den Gesamtvorgang an unter der Annahme, dass parallel durchführbare Vorgänge die Dauer der einzelnen Vorgänge nicht verlängert. Das Rezept hat die folgenden 9 Schritte und die Abhängigkeiten sind wie folgt gegeben.

    1. Zutaten abwiegen: Muss als erstes gemacht werden, keine Abhängigkeiten. Dauer: 5 Minuten.
    2. Zutaten mischen: Abhängig von "Zutaten abwiegen". Dauer: 4 Minuten
    3. Teig kneten: Abhängig von "Zutaten mischen". Dauer: 10 Minuten
    4. Backform vorbereiten: Kann unabhängig von den ersten Schritten durchgeführt werden. Dauer: 1 Minute
    5. Teig in die Backform füllen: Abhängig von "Teig kneten" und "Backform vorbereiten". Dauer: 1 Minute
    6. Ofen vorheizen: Kann parallel zu anderen Schritten erfolgen, aber muss vor dem Backen abgeschlossen sein. Dauer: 22 Minuten
    7. Kuchen backen: Abhängig von "Teig in die Backform füllen" und "Ofen vorheizen". Dauer: 20 Minuten
    8. Kuchen abkühlen lassen: Abhängig von "Kuchen backen". Dauer: 30 Minuten.
    9. Kuchen verzieren: Abhängig von "Kuchen abkühlen lassen". Dauer: 8 Minuten.

    (a) Graph:

    (b) Bestimmen Sie die minimale Gesamtdauer des Arbeitsablaufs:

13. Breitensuche und Tiefensuche im Vergleich

    Breitensuche (BFS) und Tiefensuche (DFS) können verwendet werden, um einen Graphen ausgehend von einem Startknoten zu traversieren.

    (a) Welche Datenstruktur wird bei der iterativen Breitensuche verwendet und welche bei der iterativen Tiefensuche? Begründen Sie, warum die jeweilige Datenstruktur zu dem Traversierungsverhalten des Algorithmus führt.

    (b) Gegeben sei ein Graph mit $|V|$ Knoten und $|E|$ Kanten, der als Adjazenzliste gespeichert ist. Welche Laufzeit haben BFS und DFS im Worst Case? Geben Sie die Laufzeit jeweils in Abhängigkeit von $|V|$ und $|E|$ in der O-Notation an.

    (c) Welche der beiden Algorithmen eignet sich dazu, in einem ungewichteten Graphen einen kürzesten Pfad hinsichtlich der Anzahl der Kanten von einem Startknoten s zu einem Zielknoten t zu bestimmen? Begründen Sie Ihre Antwort.

    (d) Angenommen, Sie möchten lediglich überprüfen, ob überhaupt ein Pfad von s nach t existiert. Kann dafür sowohl BFS als auch DFS verwendet werden? Begründen Sie Ihre Antwort.

14. Welche Voraussetzung muss ein Graph erfüllen, damit der Dijkstra-Algorithmus korrekt angewendet werden kann?

Dieses Übungsblatt ist Teil der Lehrveranstaltung *„Algorithmik“*.